Krishu bhati cs 17

Data structure programs

Q1 **Write a menu-based program to take the input from the user to print the following: Even numbers 1 to 10 , Prime numbers 1 to 10 and Factorial of a number.**

```
#include <stdio.h>

int main() {
   int choice, i, num, n, fact = 1, flag;

   printf("MENU\n");
   printf("1. Even numbers from 1 to 10\n");
   printf("2. Prime numbers from 1 to 10\n");
   printf("3. Factorial of a number\n");

   printf("Enter your choice: ");
   scanf("%d", &choice);

   if (choice == 1) {
      for (i = 1; i <= 10; i++) {
         if (i % 2 == 0) {
            printf("%d ", i);
         }
      }
   }

   else if (choice == 2) {
      for (num = 2; num <= 10; num++) {
         flag = 1;
         for (i = 2; i < num; i++) {
            if (num % i == 0) {
               flag = 0;
               break;
            }
         }
         if (flag == 1) {
            printf("%d ", num);
         }
      }
   }

   else if (choice == 3) {
```

```
        printf("Enter number: ");
        scanf("%d", &n);

        for (i = 1; i <= n; i++) {
            fact = fact * i;
        }

        printf("Factorial is: %d", fact);
    }

    else {
        printf("Invalid choice");
    }

    return 0;
}
```

Output:

```
MENU
1. Even numbers from 1 to 10
2. Prime numbers from 1 to 10
3. Factorial of a number
Enter your choice: 1
2 4 6 8 10

=== Code Execution Successful ===
```

```
MENU
1. Even numbers from 1 to 10
2. Prime numbers from 1 to 10
3. Factorial of a number
Enter your choice: 2
2 3 5 7

=== Code Execution Successful ===
```

Q2**Write a menu-based program to take the input from the user and print the following:Reverse of a number , Whether the number is Armstrong or not and Whether the year is leap or not.**

```
#include <stdio.h>

int main() {
    int choice;

    printf("MENU\n");
    printf("1. Reverse of a number\n");
    printf("2. Armstrong number\n");
    printf("3. Leap year\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int num, rev = 0;
        printf("Enter number: ");
        scanf("%d", &num);

        while (num > 0) {
            rev = rev * 10 + num % 10;
            num = num / 10;
        }

        printf("Reverse is: %d", rev);
```

```
}

else if (choice == 2) {
    int num, temp, d, s = 0;
    printf("Enter number: ");
    scanf("%d", &num);

    temp = num;

    while (temp > 0) {
        d = temp % 10;
        s = s + d * d * d;   // For 3-digit Armstrong
        temp = temp / 10;
    }

    if (s == num)
        printf("Armstrong number");
    else
        printf("Not an Armstrong number");
}

else if (choice == 3) {
    int year;
    printf("Enter year: ");
    scanf("%d", &year);

    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
        printf("Leap year");
```

```
        else
            printf("Not a leap year");
    }

    else {
        printf("Invalid choice");
    }

    return 0;
}
```

Output:

```
MENU
1. Reverse of a number
2. Armstrong number
3. Leap year
Enter your choice: 1
Enter number: 432
Reverse is: 234

=== Code Execution Successful ===
```

```
MENU
1. Reverse of a number
2. Armstrong number
3. Leap year
Enter your choice: 2
Enter number: 162
Not an Armstrong number

=== Code Execution Successful ===
```

Q3 **A program to perform insertion, deletion, and printing in an array.**

```
#include <stdio.h>

int main() {
   int arr[100], n = 0;
   int choice, x, i, pos, found;

   printf("1. Insert\n");
   printf("2. Delete\n");
   printf("3. Print\n");

   printf("Enter choice: ");
   scanf("%d", &choice);

   if (choice == 1) {
      printf("Enter element: ");
      scanf("%d", &x);

      arr[n] = x;
      n++;

      printf("Array: ");
      for (i = 0; i < n; i++)
         printf("%d ", arr[i]);
```

```
}

else if (choice == 2) {
   printf("Enter element to delete: ");
   scanf("%d", &x);

   found = 0;

   for (i = 0; i < n; i++) {
      if (arr[i] == x) {
         pos = i;
         found = 1;
         break;
      }
   }

   if (found == 1) {
      for (i = pos; i < n - 1; i++)
         arr[i] = arr[i + 1];
      n--;
   } else {
      printf("Element not found\n");
   }

   printf("Array: ");
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);
}

else if (choice == 3) {
   printf("Array: ");
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);
}

else {
   printf("Invalid choice");
}

return 0;
```

}

Output:

```
1. Insert
2. Delete
3. Print
Enter choice: 1
Enter element: 50
Array: 50

=== Code Execution Successful ===
```

Q4 **Write a program to find the arithmetic operations on matrices:**

Sum and Subtraction, Product of 2 matrices , Transpose of a matrix.

```
#include <stdio.h>

int main() {
    int A[2][2] = {{1,2},{3,4}};
    int B[2][2] = {{5,6},{7,8}};
    int C[2][2];
    int i, j, k;

    // Addition
    printf("Addition:\n");
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            printf("%d ", A[i][j] + B[i][j]);
        }
        printf("\n");
    }

    // Subtraction
    printf("Subtraction:\n");
```

```
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            printf("%d ", A[i][j] - B[i][j]);
        }
        printf("\n");
    }

    // Proper Matrix Multiplication
    printf("Multiplication:\n");
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            C[i][j] = 0;
            for(k=0;k<2;k++){
                C[i][j] += A[i][k] * B[k][j];
            }
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }

    // Transpose of A
    printf("Transpose of Matrix A:\n");
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            printf("%d ", A[j][i]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Output
Addition:
6 8
10 12
Subtraction:
-4 -4
-4 -4
Multiplication:
19 22
43 50
Transpose of Matrix A:
1 3
2 4


=== Code Execution Successful ===
```

Q5 **Find the element in array by using linear search.**

```
#include <stdio.h>

int main() {
   int a[5], find, i, flag = 0;

   for(i = 0; i < 5; i++){
      printf("Enter at %d : ", i+1);
      scanf("%d", &a[i]);
   }

   printf("You want to find: ");
   scanf("%d", &find);

   for(i = 0; i < 5; i++){
      if(a[i] == find){
         printf("%d is found at position %d\n", find, i+1);
         flag = 1;
         break;
```

```
        }
    }

    if(flag == 0){
        printf("Element not found");
    }

    return 0;
}
```

Output:

```
Output
Enter at 1 : 1
Enter at 2 : 10
Enter at 3 : 1000
Enter at 4 : 10000
Enter at 5 : 100000
You want to find: 1000
1000 is found at position 3


=== Code Execution Successful ===
```

Q6 Find the element in array by using binary search

```
#include <stdio.h>

int bs(int a[], int size, int found) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a[mid] == found) {
```

```
            return mid;
        } else if (found > a[mid]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    int a[5], findbro = 0;
    int size = sizeof(a) / sizeof(a[0]);

    printf("Enter 5 elements in sorted order:\n");
    for (int i = 0; i < size; i++) {
        printf("Enter element at %d place: ", i);
        scanf("%d", &a[i]);
    }

    printf("Which number to find: ");
    scanf("%d", &findbro);

    int res = bs(a, size, findbro);

    if (res == -1) {
        printf("Unable to find it");
    } else {
        printf("Found it, %d at index %d", findbro, res);
    }

    return 0;
}
```

Output:

```
Output
Enter 5 elements in sorted order:
Enter element at 0 place: 10
Enter element at 1 place: 20
Enter element at 2 place: 30
Enter element at 3 place: 40
Enter element at 4 place: 50
Which number to find: 30
Found it, 30 at index 2

=== Code Execution Successful ===
```

Q7 Infix to postfix

```
#include <stdio.h>
#include <ctype.h>

char stack[100];
int top = -1;

void push(char ch) {
   stack[++top] = ch;
}

char pop() {
   return stack[top--];
}

int precedence(char ch) {
   if (ch == '+' || ch == '-') return 1;
   if (ch == '*' || ch == '/') return 2;
   return 0;
}

int main() {
   char infix[100], postfix[100];
   int i = 0, j = 0;

   printf("Enter Infix Expression: ");
   scanf("%s", infix);
```

```
    while (infix[i] != '\0') {
        if (isalnum(infix[i])) {
            postfix[j++] = infix[i];
        }
        else if (infix[i] == '(') {
            push(infix[i]);
        }
        else if (infix[i] == ')') {
            while (top != -1 && stack[top] != '(')
                postfix[j++] = pop();
            pop();
        }
        else {
            while (top != -1 && precedence(stack[top]) >= precedence(infix[i]))
                postfix[j++] = pop();
            push(infix[i]);
        }
        i++;
    }

    while (top != -1)
        postfix[j++] = pop();

    postfix[j] = '\0';

    printf("Postfix Expression: %s", postfix);

    return 0;
}
```

Output:

```
Output

Enter Infix Expression: A+B*C-D
Postfix Expression: ABC*+D-

=== Code Execution Successful ===
```

Q8 write a program to perform all operation for the queue using the array .

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

int isFull() {
   if (rear == MAX - 1)
      return 1;
   return 0;
}

int isEmpty() {
   if (front == -1 || front > rear)
      return 1;
   return 0;
}

void enqueue(int value) {
   if (isFull()) {
      printf("Queue is Full! Cannot insert %d\n", value);
      return;
   }

   if (front == -1)
      front = 0;

   rear++;
   queue[rear] = value;
   printf("%d inserted into queue\n", value);
}

void dequeue() {
```

```
    if (isEmpty()) {
        printf("Queue is Empty! Cannot delete\n");
        return;
    }

    printf("%d deleted from queue\n", queue[front]);
    front++;
}

void peek() {
    if (isEmpty()) {
        printf("Queue is Empty!\n");
        return;
    }

    printf("Front element is: %d\n", queue[front]);
}

// Function to display queue
void display() {
    if (isEmpty()) {
        printf("Queue is Empty!\n");
        return;
    }

    printf("Queue elements are: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Queue Operations ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
```

```
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Output:

```
Output
4. Display
5. Exit
Enter your choice: 1
Enter value to insert: 30
30 inserted into queue

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements are: 10 20 30

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

Q9 write a program to perform all operation for the circular queue using the array.

```
#include <stdio.h>
#define SIZE 5
int queue [SIZE];
int front = -1, rear = -1;

void enqueue (){
   int value, i, n;
   printf ("total elements wants to be inserted: ");
   scanf ("%d", &n);

   for (i=0; i<n; i++){
```

```
        if ((rear + 1) % SIZE == front) {
            printf ("circular queue is overflow!\n");
            return;
        }
        printf ("enter value: ");
        scanf ("%d", &value);

        if (front == -1)
        front = 0;

        rear = (rear + 1) % SIZE;
        queue[rear] = value;

        printf ("value inserted! %d\n");
    }
}

void dequeue() {
    if (front == -1) {
        printf("Circular Queue Underflow! \n");
    } else {
        printf("Deleted: %d\n", queue[front]);

        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % SIZE;
        }
    }
}

void peek() {
    if (front == -1) {
        printf("Circular Queue is Empty!\n");
    } else {
        printf("Front element is: %d\n", queue[front]);
    }
}

void display() {
```

```
    int i;
    if (front == -1) {
        printf("Circular Queue is Empty! \n");
    } else {
        printf("Queue elements: ");

        i = front;
        while (1) {
            printf("%d ", queue[i]);
            if (i == rear)
                break;
            i = (i + 1) % SIZE;
        }
        printf("\n");
    }
}

int main() {
    int choice;

    while (1) {
        printf("QUEUE MENU:-\n");
        printf("1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

Output:

Output

```
QUEUE MENU:-
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
total elements wants to be inserted: 3
enter value: 5
value inserted! 5
enter value: 10
value inserted! 10
enter value: 15
value inserted! 15
QUEUE MENU:-
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
Queue elements: 5 10 15
QUEUE MENU:-
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```

Q 10 Write a program to perform all operations for the Stack using an Array.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int stack[MAX];
int top = -1;

// Push operation
void push(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow\n");
   } else {
      top++;
      stack[top] = value;
      printf("%d pushed into stack\n", value);
   }
}

void pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
   } else {
      printf("%d popped from stack\n", stack[top]);
      top--;
   }
}
void peek() {
   if (top == -1) {
      printf("Stack is empty\n");
   } else {
      printf("Top element is %d\n", stack[top]);
   }
}

void display() {
   if (top == -1) {
      printf("Stack is empty\n");
```

```
    } else {
        printf("Stack elements are:\n");
        for (int i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Stack Operations ---\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:
```

```
            printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Output

```
--- Stack Operations ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed into stack

--- Stack Operations ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 30
30 pushed into stack
```

Q11 Write a program to evaluate the Postfix Notation.

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;


void push(int value) {
   stack[++top] = value;
}


int pop() {
   return stack[top--];
}


int evaluatePostfix(char exp[]) {
   int i = 0;
   int a, b;

   while (exp[i] != '\0') {


      if (isdigit(exp[i])) {
         push(exp[i] - '0');
      }

      else {
         b = pop();
         a = pop();

         switch (exp[i]) {
            case '+': push(a + b); break;
            case '-': push(a - b); break;
            case '*': push(a * b); break;
            case '/': push(a / b); break;
```

```
        }
      }
      i++;
    }
    return pop();
}

int main() {
    char exp[MAX];

    printf("Enter postfix expression (single-digit operands): ");
    scanf("%s", exp);

    int result = evaluatePostfix(exp);
    printf("Result = %d\n", result);

    return 0;
}
```

Output

```
Enter postfix expression (single-digit operands): 82/3-
Result = 1


=== Code Execution Successful ===
```

Q12 Write a program to perform the operations of the linked list: - Insertion, - Deletion, -Display

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
    int data;
    struct Node* next;
};
```

```
struct Node* head = NULL;


void insert(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = head;
   head = newNode;
   printf("%d inserted\n", value);
}


void delete(int value) {
   struct Node *temp = head, *prev = NULL;

   if (temp != NULL && temp->data == value) {
      head = temp->next;
      free(temp);
      printf("%d deleted\n", value);
      return;
   }

   while (temp != NULL && temp->data != value) {
      prev = temp;
      temp = temp->next;
   }

   if (temp == NULL) {
      printf("Value not found\n");
      return;
   }

   prev->next = temp->next;
   free(temp);
   printf("%d deleted\n", value);
}


void display() {
```

```
    struct Node* temp = head;

    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}


int main() {
    int choice, value;

    while (1) {
        printf("\n--- Linked List Operations ---\n");
        printf("1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insert(value);
                break;

            case 2:
                printf("Enter value to delete: ");
                scanf("%d", &value);
                delete(value);
                break;

            case 3:
                display();
```

```
            break;

        case 4:
            exit(0);

        default:
            printf("Invalid choice\n");
    }
  }

  return 0;
}
```

Output

```
--- Linked List Operations ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 5
5 inserted

--- Linked List Operations ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted
```

Q13Write a program to perform the operations of the Circular linked list: - Insertion, - Deletion, -Display

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
   int data;
   struct Node* next;
};

struct Node* last = NULL;


void insert(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;


   if (last == NULL) {
      last = newNode;
      newNode->next = newNode;
   } else {
      newNode->next = last->next;
      last->next = newNode;
   }

   printf("%d inserted\n", value);
}


void delete(int value) {
   if (last == NULL) {
      printf("List is empty\n");
      return;
   }

   struct Node *temp = last->next, *prev = last;
```

```
    if (last->data == value && last->next == last) {
        free(last);
        last = NULL;
        printf("%d deleted\n", value);
        return;
    }


    if (temp->data == value) {
        last->next = temp->next;
        free(temp);
        printf("%d deleted\n", value);
        return;
    }


    prev = temp;
    temp = temp->next;

    while (temp != last->next) {
        if (temp->data == value) {
            prev->next = temp->next;

            if (temp == last)
                last = prev;

            free(temp);
            printf("%d deleted\n", value);
            return;
        }
        prev = temp;
        temp = temp->next;
    }

    printf("Value not found\n");
}


void display() {
    if (last == NULL) {
```

```
        printf("List is empty\n");
        return;
    }

    struct Node* temp = last->next;

    printf("Circular Linked List: ");
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != last->next);

    printf("(back to start)\n");
}


int main() {
    int choice, value;

    while (1) {
        printf("\n--- Circular Linked List ---\n");
        printf("1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insert(value);
                break;

            case 2:
                printf("Enter value to delete: ");
                scanf("%d", &value);
                delete(value);
                break;

            case 3:
                display();
```

```
            break;

        case 4:
            exit(0);

        default:
            printf("Invalid choice\n");
    }
  }

  return 0;
}
```

Output

```
--- Circular Linked List ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 10
10 inserted

--- Circular Linked List ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 15
15 inserted
```

Q14 Write a program to perform all operations for Circular Queue using Linked List

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
  int data;
```

```
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;


void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    if (rear == NULL) {
        front = rear = newNode;
        rear->next = front;
    } else {
        rear->next = newNode;
        rear = newNode;
        rear->next = front;
    }

    printf("%d inserted\n", value);
}


void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct Node* temp = front;


    if (front == rear) {
        printf("%d deleted\n", front->data);
        front = rear = NULL;
    } else {
        printf("%d deleted\n", front->data);
        front = front->next;
        rear->next = front;
    }
```

```
    free(temp);
}


void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Node* temp = front;

    printf("Circular Queue: ");
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != front);

    printf("(front)\n");
}


int main() {
    int choice, value;

    while (1) {
        printf("\n--- Circular Queue (Linked List) ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
```

```
            dequeue();
            break;

        case 3:
            display();
            break;

        case 4:
            exit(0);

        default:
            printf("Invalid choice\n");
    }
  }

  return 0;
}
```

Output

```
--- Circular Queue (Linked List) ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value: 50
50 inserted

--- Circular Queue (Linked List) ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
50 deleted
```

Q 15 Write a program to perform the operations of the Double linked list: - Insertion, - Deletion, -Display

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
   int data;
   struct Node* next;
};

struct Node *front = NULL, *rear = NULL;


void enqueue(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;

   if (rear == NULL) {
      front = rear = newNode;
      rear->next = front;
   } else {
      rear->next = newNode;
      rear = newNode;
      rear->next = front;
   }

   printf("%d inserted\n", value);
}


void dequeue() {
   if (front == NULL) {
      printf("Queue Underflow\n");
      return;
   }

   struct Node* temp = front;
```

```
    if (front == rear) {
        printf("%d deleted\n", front->data);
        front = rear = NULL;
    } else {
        printf("%d deleted\n", front->data);
        front = front->next;
        rear->next = front;
    }

    free(temp);
}


void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Node* temp = front;

    printf("Circular Queue: ");
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != front);

    printf("(front)\n");
}


int main() {
    int choice, value;

    while (1) {
        printf("\n--- Circular Queue (Linked List) ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Output

```
--- Circular Queue (Linked List) ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value: 30
30 inserted

--- Circular Queue (Linked List) ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
30 deleted
```

Q 16 Write a program to perform all operations for the Stack using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

struct node *top = NULL;

void push(int value) {
   struct node *newNode = (struct node*)malloc(sizeof(struct node));
   newNode->data = value;
   newNode->next = top;
   top = newNode;
}
void pop() {
   if(top == NULL) {
      printf("Stack Underflow\n");
      return;
   }

   struct node *temp = top;
   printf("Deleted element: %d\n", top->data);
   top = top->next;
   free(temp);
}

void display() {
   struct node *temp = top;

   if(temp == NULL) {
      printf("Stack is empty\n");
      return;
   }

   printf("Stack elements:\n");
   while(temp != NULL) {
```

```
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

    while(1) {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\n");
        printf("Enter choice: ");
        scanf("%d",&choice);

        switch(choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d",&value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Output

```
1.Push
2.Pop
3.Display
4.Exit
Enter choice: 1
Enter value: 5

1.Push
2.Pop
3.Display
4.Exit
Enter choice: 10
Invalid choice
```

Q17 Write a program to perform all operations for the Queue using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void enqueue(int value) {
   struct node *newNode = (struct node*)malloc(sizeof(struct node));
   newNode->data = value;
   newNode->next = NULL;

   if(rear == NULL) {
      front = rear = newNode;
```

```
    }
    else {
        rear->next = newNode;
        rear = newNode;
    }
}

void dequeue() {
    if(front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct node *temp = front;
    printf("Deleted element: %d\n", front->data);
    front = front->next;

    if(front == NULL)
        rear = NULL;

    free(temp);
}

void display() {
    struct node *temp = front;

    if(temp == NULL) {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements:\n");
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
```

```
    int choice, value;

    while(1) {
        printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
        printf("Enter choice: ");
        scanf("%d",&choice);

        switch(choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d",&value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Output

```
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 1
Enter value: 100

1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 2
Deleted element: 100
```